

APIs externes utilisées par le back-end

En bref

Cette documentation recense **toutes les APIs tierces** appelées par le back-end du projet. On distingue :

- les **APIs de données** (météo, niveaux d'eau, géocodage) — elles alimentent les modèles et les cartes ;
- les **APIs de notification** (email, SMS) — elles servent à alerter les utilisateurs.

Pour chaque API, on indique le **rôle**, l'**endpoint**, la **clé éventuelle**, le **fichier qui l'appelle** et les **limites** importantes.

Comment lire ce document

Section	De quoi ça parle
Vue d'ensemble	Tableau récapitulatif de toutes les APIs
APIs temps réel	Détail des trois APIs qui fournissent des données qui changent en continu
APIs météo (Open-Meteo)	Les 3 endpoints Open-Meteo utilisés et ce qu'on en fait
APIs Environnement Canada	GeoMet WMS et OGC API
API des niveaux d'eau (IWLS)	Détail de l'API Pêches et Océans Canada
APIs géocodage	Mapbox + Nominatim (nom de lieu à partir d'une lat/lng)
APIs de notification	SendGrid (email), Twilio (SMS)
Variables d'environnement	Toutes les clés à configurer
Cache et résilience	Comment le back-end économise les appels et réagit aux pannes

Vue d'ensemble

API	Rôle	Endpoint de base	Clé ?	Fichier appelant
-----	------	------------------	-------	------------------

Open-Meteo — Forecast	Prévisions météo 7 j (T°, pluie, neige, humidité, point de rosée)	https://api.open-meteo.com/v1/forecast	—	providers/open_meteo.py
Open-Meteo — Archive	Météo historique (cumuls, variations, entraînement)	https://archive-api.open-meteo.com/v1/archive	—	providers/open_meteo.py
Open-Meteo — Flood	Prévisions de débit fluvial	https://flood-api.open-meteo.com/v1/flood	—	services/ water_levels_service.py
IWLS (DFO-MPO)	Niveaux d'eau en temps réel (stations marégraphiques fleuve + rivières)	https://api-iwls.dfo-mpo.gc.ca/api/v1/stations	—	services/ water_levels_service.py
ECCC GeoMet — WMS	Couches raster officielles (T°, précip., humidité) sur la carte + secours météo	https://geo.weather.gc.ca/geomet/	—	services/ weather_service.py
ECCC GeoMet — OGC API	Stations climatiques canadiennes (données quotidiennes)	https://api.weather.gc.ca/collections/climate-daily/items	—	services/ weather_service.py
Mapbox — Geocoding	<i>Reverse geocoding</i> : lat/ lng → nom de quartier	https://api.mapbox.com/geocoding/v5/mapbox.places	✓	services/ place_label_service.py
Nominatim (OSM)	Fallback gratuit au reverse geocoding	https://nominatim.openstreetmap.org/reverse	—	services/ place_label_service.py
SendGrid	Envoi des alertes email	https://api.sendgrid.com/v3/mail/send	✓	alerts.py
Twilio	Envoi des alertes SMS (via SDK officiel)	SDK <code>twilio.rest.Client</code>	✓	alerts.py

APIs temps réel

Parmi toutes ces APIs, **trois** fournissent des données qui changent en continu et que le back-end ré-interroge à chaque requête utilisateur :

API	Ce qu'elle fournit	Fréquence de rafraîchissement côté source
Open-Meteo Forecast	Prévisions météo 7 j pour n'importe quelle coordonnée	Plusieurs fois par jour
IWLS (Pêches et Océans Canada)	Niveaux d'eau réels des stations marégraphiques du fleuve Saint-Laurent et des rivières	Toutes les heures (résolution SIXTY_MINUTES)
ECCC GeoMet (WMS)	Couches raster officielles de température, précipitations 24 h, humidité	Environ toutes les 6 h (cycles GDPS)

S'y ajoute **Open-Meteo Flood** pour les prévisions de débit fluvial, utilisée uniquement par le pipeline crues.

☀ Ces trois APIs suffisent pour que le logiciel fonctionne sans base de données locale pour les données dynamiques : tout est récupéré à la volée et mis en cache quelques minutes ou quelques heures (voir *Cache et résilience*).

APIs météo — Open-Meteo

Open-Meteo est un agrégateur libre qui combine plusieurs modèles météorologiques internationaux. Le projet utilise **trois endpoints**, tous interrogés via le fuseau `America/Montreal`.

1. Open-Meteo Forecast

- **URL** : `https://api.open-meteo.com/v1/forecast`
- **Rôle** : prévisions journalières et horaires jusqu'à 7-8 jours dans le futur.
- **Utilisé par** : tous les aléas.
- **Variables demandées** :

Méthode Python	Paramètre <code>daily</code> / <code>hourly</code>	Usage
<code>get_weather_forecast</code>	<code>temperature_2m_mean</code> , <code>precipitation_sum</code>	Pluvial, fluvial

get_heatwave_forecast	temperature_2m_max, temperature_2m_min, relative_humidity_2m_max + dew_point_2m horaire	Canicule (humidex calculé localement)
get_snow_forecast	temperature_2m_mean, temperature_2m_min, precipitation_sum, snowfall_sum, relative_humidity_2m_max	Neige
get_today_hourly_precipitation	precipitation, temperature_2m heures	Graphe horaire d'aujourd'hui

2. Open-Meteo Archive

- **URL** : <https://archive-api.open-meteo.com/v1/archive>
- **Rôle** : météo **historique** jusqu'à aujourd'hui-1, sur n'importe quelle plage de dates.
- **Utilisé par** : pluvial (calcul des cumuls 3/5/7 jours), fluvial (cumul 7 j + température moyenne 5 j).
- **Variables** : temperature_2m_mean, precipitation_sum.

3. Open-Meteo Flood

- **URL** : <https://flood-api.open-meteo.com/v1/flood>
- **Rôle** : prévisions de **débit fluvial** (m³/s) sur 16 jours max.
- **Utilisé par** : crues — le débit est ensuite converti en niveau d'eau via la relation simplifiée de Manning ($H_2/H_1 \approx \sqrt{(Q_2/Q_1)}$) autour d'une observation IWLS récente.
- **Variable** : daily: river_discharge.

Timeouts et limites

- Timeout HTTP côté client : **10 s**.
- Open-Meteo n'impose pas de clé ni de quota strict pour l'usage modéré (documentation publique : ~10 000 appels/jour).
- En cas d'erreur ou de réponse vide, un fallback bascule sur les couches ECCC GeoMet pour la météo courante.

APIs Environnement Canada (ECCC GeoMet)

ECCC publie ses données météo publiques via [GeoMet](#). Le projet utilise deux interfaces.

1. GeoMet WMS

- **URL** : `https://geo.weather.gc.ca/geomet/`
- **Rôle** : tuiles raster officielles affichées sur les cartes du front, et source de **secours** côté back-end si Open-Meteo tombe.
- **Appel type** : requête `GetFeatureInfo` en WMS 1.3.0 pour extraire la valeur d'un pixel à une coordonnée donnée.
- **Couches utilisées** :

Variable	Couche	Source
Température	GDPS.ETA_TT	Modèle global GDPS
Précipitations 24 h	RDPA.24F_PR	Analyse régionale
Humidité	GDPS.ETA_HR	Modèle global GDPS

2. GeoMet OGC API — `climate-daily`

- **URL** : `https://api.weather.gc.ca/collections/climate-daily/items`
- **Rôle** : fournir les **observations quotidiennes** des stations climatiques canadiennes (températures moyennes, précipitations totales) pour une bbox et une plage de dates.
- **Utilisé par** : fallback de l'historique météo si Open-Meteo Archive échoue.
- **Paramètres clés** : `bbox`, `datetime`, `limit=500`, `f=json`.

API des niveaux d'eau — IWLS (Pêches et Océans Canada)

- **URL** : `https://api-iwls.dfo-mpo.gc.ca/api/v1/stations`
- **Rôle** : niveaux d'eau observés en temps réel sur les stations marégraphiques canadiennes.
- **Utilisé par** : le modèle de **crues fluviales** (la variable `Water_Level`, la plus importante du modèle avec 61 % de poids, vient d'ici).

Endpoints concrètement appelés

Endpoint	Usage	Paramètres
<code>GET /stations</code>	Liste de toutes les stations (pour trouver la plus proche d'une coordonnée)	—

GET /stations/{id}/data	Données d'une station sur une fenêtre temporelle	time-series-code=wlo, resolution=SIXTY_MINUTES, from, to
-------------------------	--	--

Stratégie d'utilisation

1. **Trouver la station la plus proche** de (lat, lng) via un filtre sur la liste complète des stations opérantes.
2. **Récupérer la dernière valeur** (fenêtre de 2 h) pour `Water_Level`.
3. **Récupérer 7 jours d'historique** (résolution horaire, puis moyenne quotidienne en Python) pour calculer `WL_change_3d`.

Stations ignorées

Certaines stations sont explicitement **blacklistées** dans `services/water_levels_service.py` (Varenes, Barrage Fryer, Saint-Jean-sur-Richelieu, Pointe-des-Cascades, Contrecoeur IOC, Lanoraie, Sorel, Lac Saint-Pierre, Summerstown) : données jugées peu fiables ou non pertinentes pour Montréal.

APIs de géocodage

Transforment une **lat/lng** en nom de lieu humain (quartier, ville). Utilisé pour étiqueter les points dans l'interface.

1. Mapbox Geocoding (principal)

- **URL** : `https://api.mapbox.com/geocoding/v5/mapbox.places/{lng},{lat}.json`
- **Clé** : `MAPBOX_API_KEY` (même clé que la carte du front).
- **Paramètres** : `language=fr, types=neighborhood, locality, place, limit=5`.
- **Sortie** : premier résultat, raccourci à 2 segments (ex. « Plateau-Mont-Royal, Montréal »).

2. Nominatim OpenStreetMap (fallback)

- **URL** : `https://nominatim.openstreetmap.org/reverse`
- **Clé** : aucune, mais un **user-Agent** est exigé par les conditions d'usage ("`VilleIA/1.0 (Polytechnique prototype; reverse geocoding)`").
- **Paramètres** : `lat, lon, format=json, addressdetails=1, accept-language=fr`.
- **Utilisé** quand `MAPBOX_API_KEY` n'est pas configurée ou que Mapbox renvoie une erreur.

⚠ Nominatim a une **politique stricte : max 1 req/s par IP**. Pour une mise en production intensive, privilégier Mapbox ou héberger une instance Nominatim locale.

APIs de notification

1. SendGrid — email

- **URL** : `https://api.sendgrid.com/v3/mail/send`
- **Rôle** : envoyer les **alertes email** aux utilisateurs abonnés.
- **Authentification** : en-tête `Authorization: Bearer {SENDGRID_API_KEY}`.
- **Variables d'env** : `SENDGRID_API_KEY`, `ALERT_EMAIL_FROM` (adresse de l'expéditeur).
- **Si non configuré** : la fonction log un avertissement et renvoie `False` sans lever d'exception (pas de plantage).

2. Twilio — SMS

- **SDK** : `twilio.rest.Client` (package Python officiel).
- **Rôle** : envoyer les **alertes SMS** aux utilisateurs qui ont choisi ce canal.
- **Variables d'env** : `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_PHONE_NUMBER`.
- **Si non configuré** : comme pour SendGrid, warning + retour `False`.

Variables d'environnement à configurer

Variable	Utilisée par	Obligatoire ?
<code>MAPBOX_API_KEY</code>	Reverse geocoding (sinon fallback Nominatim)	✗ (recommandée)
<code>SENDGRID_API_KEY</code>	Alertes email	✗ (si pas d'emails)
<code>ALERT_EMAIL_FROM</code>	Adresse expéditeur des emails	✗ (requis si SendGrid actif)
<code>TWILIO_ACCOUNT_SID</code>	SMS	✗ (si pas de SMS)
<code>TWILIO_AUTH_TOKEN</code>	SMS	✗ (si pas de SMS)
<code>TWILIO_PHONE_NUMBER</code>	Numéro d'envoi SMS	✗ (requis si Twilio actif)

Les autres APIs (Open-Meteo, IWLS, ECCC GeoMet, Nominatim) **ne demandent aucune clé**.

Cache et résilience

Politique de cache

Pour éviter de marteler les APIs externes (et respecter les quotas publics), le back-end met certains résultats en cache :

Donnée	Durée du cache	Implémentation
Liste des stations IWLS actives	24 h	Cache global dans <code>water_levels_service.py</code>
Météo courante par coordonnée (temp, précip., humidité)	5 min	Cache dict <code>_current_weather_cache</code> dans <code>weather_service.py</code>
Historique météo par coordonnée (arrondi à 2 décimales)	durée de vie du processus	<code>@lru_cache(maxsize=256)</code> dans <code>weather_service.py</code>

Stratégie de fallback

Le back-end est conçu pour **ne pas tomber si une API tierce est temporairement indisponible** :

Échec de...	Bascule vers...
Open-Meteo Forecast (météo courante)	ECCC GeoMet WMS (<code>get_feature_info</code>)
Open-Meteo Archive (historique)	ECCC GeoMet OGC API (<code>climate-daily</code>)
Open-Meteo Flood (débit)	Forecast « plat » basé sur le dernier niveau d'eau IWLS
Mapbox Geocoding	Nominatim OSM
IWLS (panne totale)	Cache 24 h des stations + logs d'erreur côté serveur

Timeouts

Tous les appels externes utilisent un **timeout court** (10 à 15 s) pour éviter de bloquer une requête utilisateur si une API tierce est lente. En cas de timeout, le fallback prend le relais.

Glossaire

- **ECCC** — Environnement et Changement climatique Canada.
- **GeoMet** — Service de données géospatiales météo d'ECCC, utilisé via WMS (tuiles) et OGC API (requêtes sur stations).
- **IWLS** — *Integrated Water Level System* de Pêches et Océans Canada ; service REST qui expose les niveaux d'eau mesurés en temps réel.
- **Nominatim** — Service public de géocodage basé sur OpenStreetMap (gratuit, rate-limité à 1 req/s).

- **OGC API** — Norme ouverte pour servir des données géographiques sous forme de features JSON paginés.
- **Reverse geocoding** — Traduction d'une paire (latitude, longitude) en un nom de lieu humain (quartier, rue, ville).
- **WMS** — *Web Map Service*, standard OGC pour servir des tuiles cartographiques ; ECCC l'utilise pour publier ses couches météo.
- **River discharge** — Débit d'un cours d'eau en m³/s. Open-Meteo Flood le prévoit ; on le convertit localement en niveau d'eau estimé.

--	--	--

--	--	--
